

Searchable Documents Project

Find in Images

Making Scanned PDFs Searchable
in Chrome's Own Viewer with On-Device OCR

Suhani Pandey

A Chrome Extension (Manifest V3)

October 1, 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Key Idea	3
1.3	Contributions	4
1.4	Report Structure	4
2	Functional Requirements (User Stories)	4
2.1	User Stories	4
2.2	Functional Requirements	5
2.3	Non-Functional Requirements	6
3	Background and Related Work	6
3.1	The Searchable-PDF Problem	6
3.2	Searchable Scans: the OCR “Sandwich”	6
3.3	Optical Character Recognition	6
3.4	PDF Libraries: PDF.js and pdf-lib	7
3.5	The Chrome Extension Platform (Manifest V3)	7
3.6	Client-Side Storage	7
3.7	Positioning Relative to Existing Approaches	7
4	Technology Stack and Integration	8
4.1	The Stack at a Glance	8
4.2	How Each Technology Is Used	8
4.3	Coordination Between Technologies	9
5	System Design and Architecture	10
5.1	Overview	10
5.2	Components	10
5.3	The Coordinate-Mapping Problem	12
5.4	Key Design Decisions	13
6	Development Process: Phased Implementation	14
6.1	Planning: The Initial Estimate	14
6.2	Execution: How the Build Actually Unfolded	14
6.3	Plan versus Reality	17
7	Implementation	18
7.1	Extension Structure and Activation	18
7.2	Download, Validation, and Identity	19
7.3	Text Detection	19
7.4	OCR Rendering and Preprocessing	19
7.5	OCR Engine Orchestration	20
7.6	Post-Processing for Findability	20
7.7	Writing the Invisible Layer	21
7.8	Caching	21
7.9	Serving and Cleaning Up the Copy	21
8	Results and Evaluation	22
8.1	Method: End-to-End Testing in Chrome	22
8.2	Functional Outcomes	22

8.3	OCR Accuracy	22
8.4	Speed and Size	23
8.5	Visual Fidelity	23
8.6	Component Complexity	24
8.7	Discussion	24
9	Limitations and Future Work	24
9.1	Limitations	24
9.2	Future Work	25
10	Conclusion	25
	References	27

Abstract

Scanned PDFs and image-based documents are ubiquitous, yet many of them carry no machine-readable text, so the browser’s familiar “find on page” (Ctrl+F) returns nothing. This report presents FIND IN IMAGES, a Chrome extension (Manifest V3) that closes this gap entirely on the user’s device *without replacing anything the user sees*. Chrome’s own PDF viewer and its own find bar remain the only interface. When the user presses a shortcut on a scanned PDF, a hidden offscreen document downloads the file, detects which pages paint images or lack text, recognizes those pages with Tesseract.js, and writes every recognized word back into a *copy* of the PDF as invisible text (PDF text rendering mode 3), sized so that the native find highlight covers the visible word. The tab is then switched to that copy, which the extension’s service worker serves from Cache Storage, so Chrome opens it in its native viewer with the same tab title. The original file is never modified, and the copy renders pixel-identically.

The report follows the project from a custom PDF.js viewer with an injected DOM text layer to this “searchable-scan sandwich” design, and explains why the change was needed: no DOM viewer can reproduce Chrome’s native PDF experience. It describes the coordinate mapping from OCR pixels into PDF user space, the OCR preprocessing (including a gain-capped contrast stretch that keeps sparse pages from collapsing into noise), content-addressed caching in IndexedDB, and resource management for a long-lived background pipeline. It closes with an end-to-end test suite that drives the packaged extension in real Chrome. That suite exposed a defect that unit-level tests had missed: Chrome refuses to navigate an ordinary web tab to an extension-owned `blob:` URL, so the first version never actually switched tabs.

1 Introduction

1.1 Motivation

Documents on the web fall broadly into two categories. *Text-based* PDFs embed their characters as selectable, searchable glyphs and work seamlessly with the browser’s built-in find feature. *Image-based* documents—scanned pages, photographs of text, screenshots, and PDFs whose text has been flattened into raster images—contain no machine-readable text at all. To the browser they are pictures. Pressing Ctrl+F on such a document finds nothing, even though the text is plainly visible to a human reader.

This is a pervasive friction point. A student searching a scanned textbook, a researcher scanning archival material, or anyone working with a faxed or photographed form repeatedly hits the same wall: the content is *there*, but not *findable*. The conventional remedy—uploading the file to a server-side OCR service to produce a new, searchable PDF—is slow, raises privacy concerns, and breaks the natural in-browser reading flow.

1.2 Key Idea

FIND IN IMAGES does not build a search engine. Instead, it *makes the document itself searchable* and leaves searching to the browser. It recovers the text baked into images with on-device OCR and writes that text back *into the PDF* as an invisible layer, the same “searchable scan” technique that document scanners use. Chrome’s own viewer then opens the result. From the browser’s perspective the document now *contains* text, so Chrome’s mature find, highlighting, selection, and copy machinery applies unchanged.

Two convictions shaped the project. The first is that *the cheapest way to make something searchable is to make it real text*, rather than intercepting and re-implementing search. The second came from the user later in the project: the extension must not present a different view at all. Chrome’s own viewer, toolbar and find bar must be the only interface, and the PDF must

look exactly as it did. The final architecture satisfies both by changing *what* Chrome displays (a searchable copy) rather than *how* it displays it.

1.3 Contributions

This report documents the design and implementation of FIND IN IMAGES. Its principal contributions are:

- A fully **client-side, offline** pipeline that turns a scanned PDF into a searchable one inside the browser, with no server round-trip and no data leaving the user’s machine.
- A **native-viewer-only delivery model**: OCR runs in an offscreen document, and the searchable copy is served from the extension’s own origin by its service worker, so Chrome’s built-in viewer displays it with the original’s tab title (Section 5).
- A **coordinate mapping** from OCR pixel boxes into PDF user space, including rotated pages. Each invisible word is sized and horizontally squeezed so that Chrome’s native find highlight covers the visible word (Section 5).
- An **OCR accuracy layer**: high-resolution rendering, gain-capped contrast stretching, sharpening, and post-processing that corrects likely digit/letter confusions (Section 7).
- **Content-addressed caching and bounded resources**: OCR results keyed by a SHA-256 hash of the file, least-recently-used pruning, and automatic release of OCR workers and searchable copies.
- An **end-to-end test suite** that loads the packaged extension into real Chrome and verifies the whole user journey (Section 8).

1.4 Report Structure

Section 2 states the system’s functional requirements as user stories. Section 3 surveys the relevant background and prior approaches. Section 4 catalogues the technologies used and how they coordinate. Section 5 presents the system architecture, the coordinate mapping, and the key design decisions. Section 6 recounts the phased development of the project, including two architectural reversals, and Section 7 details the implementation of each pipeline stage. Section 8 evaluates the system, Section 9 discusses limitations and future work, and Section 10 concludes.

2 Functional Requirements (User Stories)

Before the architecture, it is worth stating plainly what the system must do for its users. FIND IN IMAGES has essentially one kind of user—anyone viewing a PDF in Chrome—and one overriding need: to find and reuse text that is visible on the page, whether or not the document stores it as real text. The requirements below are written first as user stories, then distilled into functional and non-functional requirements that the rest of the report can be traced against.

2.1 User Stories

- **Find anything visible.** *As a reader of a scanned PDF, I want Ctrl+F to find words that appear only inside images, so that no visible text is unsearchable.*
- **Nothing looks different.** *As a user, I want to keep Chrome’s own PDF viewer, toolbar, and find bar, and I want the PDF to look exactly as it did, with no new viewer to learn.*

- **One gesture, reversible.** *As a user, I want one shortcut or click to make the open PDF searchable, and the same gesture to bring the original back.*
- **Mixed pages.** *As a reader of a page that mixes real text and text baked into images, I want both to be searchable, without a word being matched twice.*
- **Select and copy.** *As a user, I want to select and copy the recovered text, so that I can quote or reuse it elsewhere.*
- **Choose the language.** *As a reader of non-English documents, I want to pick the OCR language (English, Danish, or both).*
- **Know what is happening.** *As a user, I want to see progress while a document is processed, and a clear reason when it cannot be (for example, the file is encrypted or the page is not a PDF).*
- **Works where my PDFs are.** *As a user, I want it to work for PDF links without “.pdf” in the address, on sites I am signed in to, and for files on my own computer.*
- **Instant on revisit.** *As a returning user, I want a document I have already processed to become searchable instantly.*
- **Private and offline.** *As a privacy-conscious user, I want everything to happen on my machine, with no upload and no network dependency.*

2.2 Functional Requirements

Table 1 restates the stories as testable functional requirements and traces each to the development phase that delivered it (Section 6). Every row is exercised by the end-to-end test suite described in Section 8.

Table 1. Functional requirements stated as testable conditions, each traced to the development phases that delivered it (Section 6).

ID	Requirement	Delivered in
FR-1	Make text inside images findable with Chrome’s native find	Phases 3, 11, 14
FR-2	Keep existing PDF text searchable	Phases 3, 14
FR-3	On mixed pages, add image text without duplicating existing text	Phases 3, 14
FR-4	Allow the recovered text to be selected and copied	Phases 10, 14
FR-5	Display only in Chrome’s native viewer; one gesture switches to the searchable copy and back	Phases 12, 14, 16
FR-6	Support multiple OCR languages (English, Danish)	Phase 8
FR-7	Place recovered text correctly on rotated pages	Phases 7, 14
FR-8	Show progress, and explain failures (encrypted PDF, not a PDF, no file access)	Phases 14, 16
FR-9	Reuse earlier results instantly, but never for a file whose contents changed	Phases 4, 16
FR-10	Accept any PDF a tab shows: links without .pdf, signed-in sites, local files	Phase 16

2.3 Non-Functional Requirements

Table 2. Non-functional requirements — system-wide qualities that apply regardless of which page or document is being processed — and the design decisions or phases that realise each one.

ID	Requirement	Realised by
NFR-1	Non-destructive, visually identical: the original file is never modified; the searchable copy renders pixel-identically	§5, Phase 14
NFR-2	Native UI only: no custom viewer, find bar, or page overlay	Phase 14
NFR-3	Offline & private: no network at runtime except re-downloading the PDF itself; bundled OCR engine and models	Phases 5, 14
NFR-4	Responsive: the tab stays usable while OCR runs in a hidden document on a worker pool	Phases 9, 14
NFR-5	Accurate: image-text recovery as close to “fully findable” as pre-processing allows	Phases 6, 11, 16
NFR-6	Bounded resources: the cache is pruned, idle OCR workers are released, and copies are deleted when no longer shown	Phase 16
NFR-7	Verified in the real browser: the packaged build passes an automated end-to-end suite in Chrome	Phase 16

3 Background and Related Work

3.1 The Searchable-PDF Problem

The PDF format [1] can represent text either as encoded glyphs with positioning operators or as raster images. Only the former is extractable. A “scanned” PDF is typically a sequence of full-page images with no text operators, so any tool relying on the document’s text stream—including the browser’s find feature—sees an empty page. Making such a document searchable requires *recovering* the text through OCR and re-associating it with the page.

3.2 Searchable Scans: the OCR “Sandwich”

The PDF format offers a standard way to do that re-association. Text drawn with *text rendering mode 3* (operator `3 Tr`) is neither filled nor stroked: it is invisible, yet it is still part of the page’s text, so viewers can find, highlight, select, and copy it [1]. Document scanners and tools such as `ocrmypdf` [2] use this to produce *searchable scans*: the original page image stays on top, and an invisible layer of recognized words sits underneath at matching positions. Because the text really is in the file, every PDF viewer can search it without knowing anything about OCR. FIND IN IMAGES produces exactly this kind of file, but inside the browser and only when the user asks.

3.3 Optical Character Recognition

OCR converts images of text into character sequences. Tesseract, originally developed at HP and later open-sourced by Google, is among the most widely used engines; modern versions use an LSTM-based recognizer and report results as a hierarchy of blocks, paragraphs, lines, and words, each with a bounding box and a confidence score [3]. FIND IN IMAGES uses Tesseract.js [4], a WebAssembly port that runs entirely in the browser, which is what makes a fully client-side, offline pipeline possible.

OCR quality is sensitive to input image quality. Low contrast, blur, and low resolution all degrade recognition, which motivates the preprocessing stage described in Section 7. Classic techniques such as Otsu’s thresholding [5] binarize an image by analyzing its gray-level histogram; FIND IN IMAGES performs a related percentile-based contrast stretch before handing the image to Tesseract’s own binarizer.

3.4 PDF Libraries: PDF.js and pdf-lib

PDF.js [6] is Mozilla’s standards-based PDF parser and renderer written in JavaScript. FIND IN IMAGES uses three of its capabilities: `getTextContent()`, which returns positioned text items for pages that *do* have a text layer; `getOperatorList()`, which exposes the drawing operators and reveals whether a page paints raster images; and `render()`, which rasterizes a page to a canvas for OCR. PDF.js also defines a *viewport* that maps PDF user space (points, bottom-left origin) to pixels, including page rotation; its inverse is the bridge from OCR pixels back into the PDF.

pdf-lib [7] is a JavaScript library for creating and *modifying* PDFs. It can append raw content-stream operators to an existing page and save the document, keeping the original content intact. FIND IN IMAGES uses it to write the invisible text layer.

3.5 The Chrome Extension Platform (Manifest V3)

Manifest V3 [8] is the current Chrome extension architecture. The components relevant here are:

- **Service worker** — an event-driven background script that wakes for the user’s gesture, tracks progress, switches the tab, and (as an ordinary service worker) can answer `fetch` events for the extension’s own URLs [9].
- **chrome.action and chrome.commands** — the toolbar button (with a per-tab badge and tooltip) and a browser-global keyboard shortcut. These are the two ways the user invokes the extension.
- **Offscreen documents** [10] — hidden extension pages with a DOM. Service workers have no canvas and cannot host the OCR pipeline, so it runs in an offscreen document.
- **A content security policy** permitting `wasn-unsafe-eval`, required so the bundled WebAssembly OCR core can execute.

Chrome’s built-in PDF viewer cannot be scripted or extended by extensions. Any approach that keeps that viewer must therefore change the *file* it displays, not the viewer itself.

3.6 Client-Side Storage

Three browser stores are used. IndexedDB [11], a transactional key–value store, caches OCR results across sessions. The Cache Storage API [12] holds the searchable copies, as HTTP responses that the service worker can return directly. Extension storage (`chrome.storage`) keeps the OCR language and which tabs are showing a copy. Document identities are SHA-256 digests computed with the Web Crypto API [13].

3.7 Positioning Relative to Existing Approaches

Server-side services (commercial PDF tools and `ocrmypdf` run on a server) produce a searchable PDF but require an upload and a round-trip, and they take the user out of the in-browser reading flow. Browser PDF viewers render scanned PDFs but cannot search them. Custom in-browser

viewers can add search, but they replace the viewer the user knows—a cost this project first paid and later removed (Section 6). FIND IN IMAGES combines the strengths: it produces a real searchable PDF like `ocrmypdf`, does so locally and privately, and shows it in the browser’s own viewer. The trade-off is that recognition runs on the client’s CPU, and the architecture in Section 5 is largely a response to making that trade-off acceptable.

4 Technology Stack and Integration

Every part of FIND IN IMAGES runs on the client, inside the browser. The system is assembled from a small set of web technologies, each chosen for one job, and the engineering value is largely in how they are *coordinated*. This section catalogues the stack, explains how each piece is used, and then shows how they fit together.

4.1 The Stack at a Glance

Table 3 lists each technology, the single job it performs, and the file where it is wired into the system; the subsections that follow expand on how each one is used.

Table 3. Technologies used in FIND IN IMAGES, the single role each plays, and the source file where it is wired into the system.

Technology	Role in the project	Where
Chrome Extension (MV3)	Packaging, permissions, and the CSP that lets WASM OCR run	<code>manifest.json</code>
Service worker + <code>chrome.action</code> / <code>chrome.commands</code>	Trigger, per-tab progress badge, tab switch, and serving the searchable copy	<code>service-worker.js</code>
Offscreen document	Hidden page with a DOM that hosts the whole pipeline	<code>offscreen/offscreen.js</code>
PDF.js	Detects text and images per page; renders pages for OCR	<code>processor.js</code> , <code>ocr.js</code>
HTML5 Canvas	OCR raster target and pixel-level preprocessing	<code>ocr.js</code>
Tesseract.js (WASM) + Web Workers	Optical character recognition on a pool of workers	<code>ocr.js</code>
pdf-lib	Writes the invisible text layer into the copy and saves it	<code>embed.js</code>
Web Crypto (SHA-256)	Content-based document identity	<code>cache.js</code>
IndexedDB	Persistent OCR result cache with LRU pruning	<code>cache.js</code>
Cache Storage	Holds searchable copies, served at extension URLs	<code>offscreen.js</code> , <code>service-worker.js</code>
<code>chrome.storage</code> , <code>chrome.contextMenus</code>	OCR language and open copies; the language menu	<code>service-worker.js</code>
Chrome’s native PDF viewer	All display, find, highlighting, selection, and printing	(Chrome)

4.2 How Each Technology Is Used

Chrome Extension (Manifest V3).

`manifest.json` requests the `offscreen`, `storage` and `contextMenus` permissions and host access to fetch PDFs, sets a content security policy of `script-src 'self' 'wasm-unsafe-`

`eval'` so Tesseract's WebAssembly core can execute, and requires Chrome 116 or later (for `chrome.runtime.getContexts`). There are no content scripts and no web-accessible resources: the extension never touches web pages.

Service worker, `chrome.action` & `chrome.commands`.

The service worker sleeps until the user clicks the toolbar icon (`action.onClicked`) or presses the shortcut (`commands.onCommand`, `Ctrl+Shift+F`; `Control+Shift+F` on Mac). It starts the offscreen document, sends it the job, mirrors progress as a per-tab badge (an ellipsis, then a percentage, then a check mark, or a red ! whose tooltip explains the error), and finally navigates the tab to the copy. Its `fetch` handler serves the copy's bytes, and invoking the extension on a copy restores the original URL.

Offscreen document.

Created with reason `WORKERS`, it provides the DOM, canvas and Web Workers the pipeline needs. It processes one PDF at a time from a queue and releases its OCR workers after a minute idle.

PDF.js.

`getDocument({data})` parses the downloaded bytes. Per page, `getTextContent()` returns existing text, `getOperatorList()` (checked against the `pdfjsLib.OPS` image operators) reveals raster images, and `render()` rasterizes pages that need OCR. `viewport.convertToPdfPoint` maps OCR pixels back into PDF user space. PDF.js never displays anything to the user.

Canvas API.

Each page needing OCR is rendered to an offscreen canvas about 3000 px wide, preprocessed through `getImageData` / `putImageData`, and released immediately after recognition.

Tesseract.js.

Loaded as a UMD script exposing Tesseract. A `createScheduler()` fronts a pool of LSTM `createWorker()` workers configured with `PSM.SPARSE_TEXT`; `addJob("recognize")` with `blocks:true` returns the block → word hierarchy with bounding boxes. The WASM core and the `eng/dan` language data are bundled, so OCR is fully offline. Only the SIMD and plain LSTM cores are shipped; the relaxed-SIMD core is disabled (Section 6).

pdf-lib.

Loads the same bytes, embeds the standard Helvetica font once, appends one invisible text object per recognized word to each page's content stream, and saves the copy with metadata unchanged.

Web Crypto, IndexedDB, Cache Storage.

`crypto.subtle.digest` computes the SHA-256 of the downloaded file; IndexedDB stores each page's recognized words under `hash::lang::page`; Cache Storage holds the finished copy as an HTTP Response, keyed by its URL path.

`chrome.storage` and `chrome.contextMenus`.

The OCR language lives in `storage.local` and is chosen from radio items on the toolbar icon's right-click menu; `storage.session` records which tab shows which copy, so the copy can be deleted when that tab closes or navigates.

4.3 Coordination Between Technologies

Figure 1 traces the data that crosses each boundary. The service worker is the coordinator between the browser and the extension; the offscreen document's `processor.js` is the coordinator

of the libraries. The two exchange only small messages—a job description going in, progress and a copy URL coming out—while the heavy bytes travel through Cache Storage, which both can reach because they share the extension’s origin.

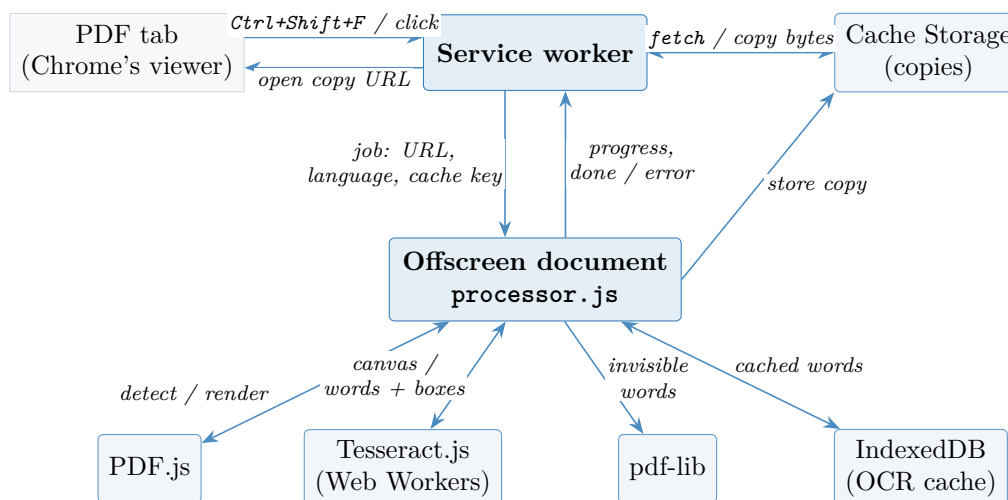


Figure 1. Integration view. The service worker talks to the browser (trigger, tab navigation, serving the copy); the offscreen document drives the libraries. Arrow labels name the data crossing each boundary; double-headed arrows indicate request–response pairs. The finished copy reaches the viewer through Cache Storage, not through messages.

5 System Design and Architecture

5.1 Overview

FIND IN IMAGES is a *pipeline* that turns a scanned PDF into a searchable copy of itself and then lets Chrome show that copy. Chrome’s native PDF viewer displays every PDF as usual. When the user invokes the extension on one (keyboard shortcut or toolbar icon), the service worker hands the job to a hidden offscreen document. The tab keeps showing the original while the offscreen document downloads the file, finds the pages that paint images or have no text, recognizes them with OCR, and writes the recognized words into a copy as invisible text. The tab is then navigated to the copy, still in Chrome’s own viewer, where Chrome’s own Ctrl+F finds the words. Figure 2 shows the end-to-end flow.

5.2 Components

Activation and coordination

The service worker responds to the toolbar icon or the keyboard shortcut, starts the offscreen document, sends it the job, shows progress on the icon’s badge, and navigates the tab when the copy is ready. Invoked on a copy, it restores the original.

Offscreen document

A hidden extension page with a DOM. It queues jobs and runs the pipeline; nothing in it is ever visible.

Download and identity

The PDF is fetched again from its URL with the user’s cookies, checked for the %PDF-signature, and identified by the SHA-256 of its bytes.

Text detection

For each page, PDF.js reports existing text and whether the page paints raster images.

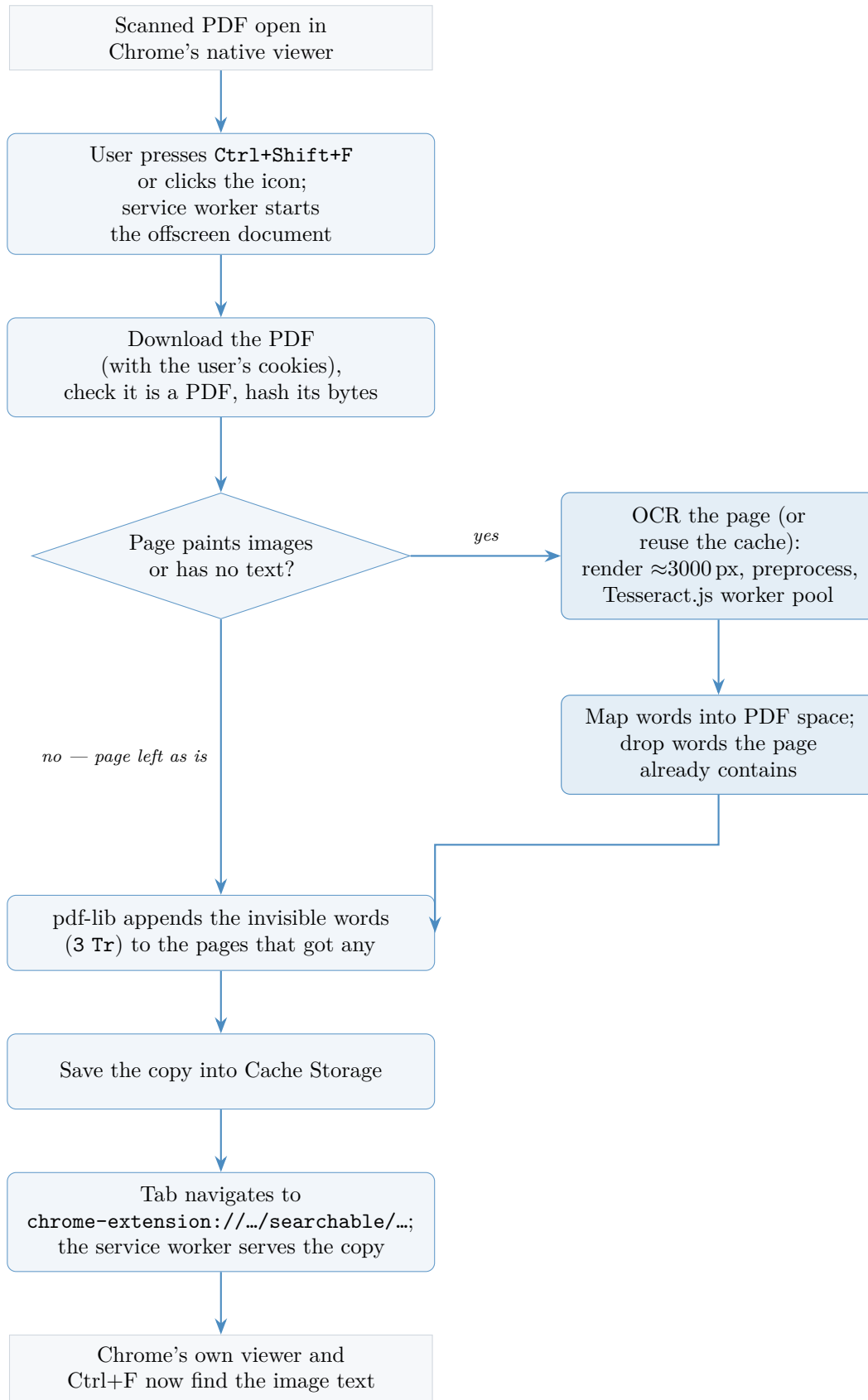


Figure 2. End-to-end processing pipeline. Every page is checked; pages that paint raster images *or* have no extractable text are OCR'd (right branch) unless the cache already holds their words, and the recognized words are mapped into the page's coordinate system with duplicates of existing text removed. pdf-lib then writes them into a copy as invisible text. The copy is shown at an extension URL in Chrome's native viewer, so the user never sees anything but Chrome's own interface.

OCR

Pages that need it are rendered offscreen at high resolution, preprocessed, and recognized by a pool of Tesseract.js workers. Results are cached in IndexedDB.

Mapping and embedding

Recognized word boxes are mapped into PDF user space, de-duplicated against existing text, and written into the page as invisible text by pdf-lib.

Delivery

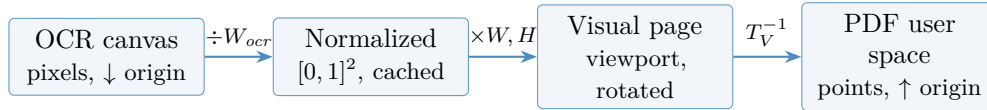
The saved copy goes into Cache Storage; the service worker’s `fetch` handler serves it at an extension URL, and Chrome’s viewer displays it.

Search

Chrome’s native find, highlighting, selection, and copy—no code of ours runs at search time.

5.3 The Coordinate-Mapping Problem

The geometric problem is to place each invisible word so that Chrome’s find highlight lands on the visible word. Chrome draws that highlight from the glyphs’ own geometry—their baseline, font size and advance widths—so the text must be written at the right position, size, angle, and width in *PDF user space*. The recognized word, however, starts life in a different space:



Tesseract reports word boxes (x_0, y_0, x_1, y_1) in pixels of the OCR canvas. These are first normalized by the canvas size,

$$\tilde{x} = \frac{x}{W_{\text{ocr}}}, \quad \tilde{y} = \frac{y}{H_{\text{ocr}}}, \quad (1)$$

so that cached results do not depend on the resolution at which OCR ran.

The OCR canvas was rendered through a PDF.js viewport, whose affine transform T_V maps PDF user space to the *visual* page (rotation and the flip from a bottom-left to a top-left origin included). Let $W \times H$ be that page’s size at scale 1. The word’s baseline is taken at 80% of its box height, and four points are mapped back with the inverse transform T_V^{-1} (PDF.js `convertToPdfPoint`):

$$\begin{aligned} y_b &= (\tilde{y}_0 + 0.8(\tilde{y}_1 - \tilde{y}_0)) H, \\ S &= T_V^{-1}(\tilde{x}_0 W, y_b), & E &= T_V^{-1}(\tilde{x}_1 W, y_b), \\ P_t &= T_V^{-1}(\tilde{x}_0 W, \tilde{y}_0 H), & P_b &= T_V^{-1}(\tilde{x}_0 W, \tilde{y}_1 H). \end{aligned} \quad (2)$$

S and E are the baseline’s start and end in PDF space, and P_t, P_b the box’s top and bottom. From them follow the word’s width, its font size, and its baseline angle:

$$w = \|E - S\|, \quad h = \|P_b - P_t\|, \quad \theta = \text{atan2}(E_y - S_y, E_x - S_x). \quad (3)$$

Because rotation is folded into T_V , rotated pages need no special case: on a page with `/Rotate 90`, θ simply comes out as a right angle.

Each word is then written as one invisible text object. With w_{nat} the word’s natural width in Helvetica at size h , the horizontal scaling (operator `Tz`, in percent) stretches or squeezes it to exactly w :

$$s = \text{clamp}\left(100 \frac{w}{w_{\text{nat}}}, 1, 1000\right), \quad T_m = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ S_x & S_y & 1 \end{bmatrix}. \quad (4)$$

The resulting content-stream fragment is `BT 3 Tr /F h Tf s Tz Tm Tm (word) Tj ET`. Font size h makes the highlight as tall as the visible word, s makes it as wide, and T_m puts it in the same place at the same angle.

Existing PDF text lives in user space already (PDF.js returns each item’s transform), so de-duplication compares like with like: an OCR word is dropped when the midpoint of its baseline, $(S + E)/2$, falls inside an existing text item’s box *and* that item’s text contains the word. Different text at the same spot—for example a caption baked into an image next to real text—is kept.

5.4 Key Design Decisions

Seven decisions shape the rest of the system. Several reverse earlier choices, and Section 6 tells how each was reached.

1. **Chrome’s viewer is the only interface: change the file, not the viewer.** Chrome renders PDFs in a built-in viewer that extensions cannot script or extend. Earlier versions of FIND IN IMAGES therefore re-rendered the PDF in a custom PDF.js viewer with an invisible DOM text layer. No DOM viewer can reproduce Chrome’s (its toolbar, thumbnails, printing, Google Lens, right-click menu, and above all its find bar), and an attempt to imitate it was rejected by the user. The final design gives Chrome a different *file* instead—one that contains the text—so every pixel of interface is Chrome’s own.
2. **Never modify the original; write into a copy.** The source file is downloaded and left untouched. Only a copy receives the invisible layer, and because mode-3 text paints nothing, the copy renders pixel-identically to the original. Invoking the extension again restores the original URL.
3. **Activate on demand.** The extension’s value is real only for scanned and image PDFs, and OCR is expensive, so nothing happens until the user asks. Ordinary PDFs keep Chrome’s default behaviour with zero overhead.
4. **Serve the copy from the extension’s origin, not from a blob: URL.** The first version opened the copy as a `blob: URL`. In real Chrome that navigation never completes: an extension’s blob URL cannot be opened from an ordinary website tab, consistent with Chrome’s partitioning of blob URLs by site [14]. The copy is therefore stored in Cache Storage and served by the service worker’s `fetch` handler at `chrome-extension://<id>/searchable/<token>/<file name>?src=<original>`. Ending the path in the original file name keeps the tab title; the `src` parameter makes restoring trivial and lets an expired copy fall back to the original instead of an error page.
5. **Run the pipeline in a hidden document.** Rendering and OCR need a DOM and Web Workers, which service workers lack, and must not disturb the tab. An offscreen document provides both, and its PDF.js rendering exists purely for OCR (Section 7).
6. **Identify documents by content.** OCR results are cached by the SHA-256 of the file’s bytes rather than its URL. The same file reached through two URLs is recognized once, and a changed file at the same URL (an edited local file, a “latest.pdf” link) can never receive stale words.
7. **Keep a long-lived background component bounded.** The offscreen document can live for the whole browser session, so every resource it creates has an owner and an end: pdf.js documents are destroyed after each job, OCR workers are released after a minute

idle, copies are deleted when their tab closes or navigates away (and at browser start-up), and the OCR cache is pruned to about 2,000 pages.

6 Development Process: Phased Implementation

FIND IN IMAGES was built incrementally, one phase at a time, with each phase shipped and tested before the next began. Two principles held from the start: the original PDF is **never modified**, and the headline capability, **finding text inside images**, had to be as close to “100% findable” as possible. A third arrived late and reshaped the whole system: the user must see **nothing but Chrome’s own viewer**. This section records how the work was planned, how it actually unfolded, and the reasoning at each step.

6.1 Planning: The Initial Estimate

Before writing code, the work was broken into seven phases with rough effort estimates (Table 4). Two were flagged up front as the likely time-sinks: the *invisible text-layer overlay* (the coordinate-alignment problem) and *intercepting Chrome’s built-in PDF viewer*, which is sandboxed and cannot simply be scripted.

Table 4. Initial seven-phase plan drawn up before implementation began. Phases 5 and 6 were flagged as the expected time-sinks; Phase 6 was ultimately pulled forward and solved first.

#	Planned phase	Note
1	Extension scaffolding (MV3)	Manifest, service worker, content-script wiring
2	PDF.js rendering pipeline	Pages to canvas; worker setup in an extension context
3	Tesseract.js OCR	Plumbing OCR into the async page pipeline
4	Per-page text detection	<code>getTextContent()</code> check — quick
5	Invisible text-layer overlay	<i>Hardest</i> : PDF↔canvas↔DOM coordinate alignment
6	Intercept Chrome’s PDF viewer	<i>Hidden complexity</i> : sandboxed viewer; must redirect
7	Polish, edge cases, testing	—

6.2 Execution: How the Build Actually Unfolded

Execution diverged from the plan in two instructive ways. First, viewer interception (planned as Phase 6) turned out to be a *prerequisite* for everything else—you cannot render or overlay anything until you control the page—so it was solved first, folded into Phase 1. Second, “OCR accuracy” was not a single step but the dominant theme of the back half of the project: it expanded into several phases as the README roadmap and real test feedback exposed new failure modes. The phases below are the build as it actually happened.

Foundation.

Phase 1 — Extension skeleton & PDF interception.

Built the MV3 manifest, service worker, content script, and an empty viewer, then made the service worker redirect `.pdf` navigations to that viewer. *Why first*: Chrome opens PDFs in its own sandboxed viewer that scripts cannot reach, so gaining control of the page is the true starting point—and it establishes the read-only-overlay architecture. *Revealed*: a viewer that shows nothing yet → Phase 2.

Phase 2 — PDF.js rendering pipeline.

Bundled PDF.js locally (to satisfy the MV3 CSP) and rendered each page to a canvas, switching to a *dynamic* redirect rule so the original URL survives as `?url=`. *Friction*: `file://` security origins and the `getDocument` URL parameter both needed care. *Revealed*: pages display but nothing is searchable → Phase 3.

Phase 3 — Text detection, OCR & the invisible overlay.

The heart of the project: detect native text with `getTextContent()`, OCR image pages with Tesseract.js in a worker, and inject invisible spans over the canvas. *Key pivot*: the first design was *either/or* (use native text, else OCR), but a page containing the same word both as native text *and* inside an image matched only the native copy—image text was lost. The design changed to *both*: always inject native text, then OCR images and merge only the non-duplicate words (text-aware de-duplication). *Revealed*: OCR re-ran on every load → Phase 4.

Phase 4 — IndexedDB caching.

Cache each page's words once, keyed per document, with coordinates normalized to `[0,1]` so the cache survives any zoom level. *Why*: OCR is the expensive step, so it should be a one-time cost. *Revealed*: language data still implied a CDN → Phase 5.

Phase 5 — Fully offline.

Bundled the LSTM language model and pointed Tesseract's `langPath` at it, removing the last network dependency. *Why*: privacy, reliability, and offline use—no document data ever leaves the machine.

Refinement (working through the README roadmap and test feedback).

Phase 6 — OCR accuracy: correction variants.

Added confidence filtering, edge-noise trimming, and digit→letter correction variants (`he11o`→`hello`) injected as search-only spans. *Thought*: native find is exact-match, so rather than fuzzy-matching at query time, widen the *stored* text.

Phase 7 — Rotated pages.

Derived each glyph's angle and size from the `viewport.transform × item.transform` matrix and rotated the spans to match, so rotation is handled by the math rather than special cases (Section 5).

Phase 8 — Multi-language OCR.

Added an English/Danish selector with a per-language cache, so switching languages reprocesses independently and non-destructively.

Phase 9 — Performance on large PDFs.

Introduced lazy, viewport-driven rendering, idle-time background indexing, and *parallel* OCR via a worker scheduler. *Thought*: a single serial OCR worker was the real bottleneck; the roadmap's "GPU acceleration" proved not actionable (Tesseract.js has no GPU path and already uses the SIMD WASM core), so parallelism was the available win.

Phase 10 — Readability & search UX.

Driven directly by user testing: the text layer was made selectable/copyable; a measure-and-scale fix aligned each invisible span to its exact width after selection and search landed on the wrong characters ("Where is"→"Where is the b"); and the native find bar—which flashed the invisible layer as oversized doubled text—was replaced by a custom find bar with translucent highlights, ↑ / ↓ navigation, and a distinct active-match style for colourful pages.

Phase 11 — OCR image preprocessing.

The final push on the “100% findable” goal, targeting the cases still failing: sparse-text segmentation recovered *tables*; a high-resolution (3000 px) OCR canvas recovered *small* text; and a percentile contrast stretch plus unsharp mask attacked *low-contrast and blurred* text (the cache format version was bumped to invalidate stale results). *Thought*: decouple the OCR image from the display image and push recognition as far as preprocessing allows—while accepting a hard ceiling on genuinely low-resolution source images.

Production (preparing for release).

Phase 12 — Activation model: from automatic to on demand.

The biggest late change reversed Phase 1’s founding decision. Until now the extension intercepted *every* PDF navigation with `declarativeNetRequest` and forced it into the custom viewer. Dogfooding on ordinary text PDFs exposed the flaw: Chrome’s native PDF viewer is a sandboxed plugin, so replacing it threw away its entire toolbar and right-click menu (thumbnails, rotate, print, download, Google Lens, ...)—none of which a DOM viewer can reproduce—while for an already-searchable text PDF the extension added nothing. *Decision*: keep Chrome’s native viewer as the default and invoke the OCR viewer *on demand* via a keyboard shortcut (`Ctrl+Shift+F`) or the toolbar icon, toggling back on a second press. *Why*: the extension’s value is real only for image and scanned PDFs, and those are precisely the documents where the native toolbar matters least—so scoping activation to them preserves the native experience everywhere else (decision 3). *Side benefit*: dropping interception also removed the `declarativeNetRequest` and `storage` permissions and the catch-all content script, leaving a much smaller, easier-to-review permission footprint.

Phase 13 — Imitating the native viewer.

Even on demand, the OCR viewer was still a *different* viewer, so the next effort made it look and behave like Chrome’s: a Chrome-style toolbar with zoom and page navigation, a find bar restyled after Chrome’s (match counter, `↑ / ↓`, `Esc/Enter/Shift+Enter`), Chrome’s yellow and orange highlights, the find bar opening already focused, and a return to the native viewer at the same page via `#page=N`. A new search engine matched phrases across OCR word boundaries (the old one matched only inside a single span), and background indexing stopped freezing in hidden tabs once rendering used PDF.js’s `print` intent, which is not paced by `requestAnimationFrame`. *Revealed*: the user’s verdict was that an imitation is not the original—they wanted “same behaviour as that of original ... and same design and view also”, with no different view and no change to the PDF’s format → Phase 14.

Phase 14 — Native viewer only: the searchable-scan sandwich.

The custom viewer was deleted outright (about 1,300 lines). In its place, an offscreen document OCRs the PDF and uses pdf-lib to write the recognized words into a *copy* as invisible text (3 Tr), each word sized and squeezed so Chrome’s highlight covers it (Section 5.3); the tab then switches to that copy in Chrome’s own viewer. Progress moved to a badge on the toolbar icon and the OCR language to the icon’s right-click menu. *Thought*: the only way to give the user Chrome’s exact interface is to give Chrome a file it can search itself. This also reinterpreted “never modify the PDF”: the original is still never touched, and the copy’s visible content was verified identical by pixel comparison (0 of 2,298,825 pixels differed). *Gap*: the pipeline was tested in a browser tab, but the final tab switch was not verified in Chrome itself.

Phase 15 — First use in the user’s browser.

Loading the build into the user’s own Chrome surfaced two problems. OCR crashed inside the offscreen document in Tesseract’s *relaxed-SIMD* WebAssembly core: Tesseract.js chooses

that core when a `WebAssembly.validate()` probe accepts it, but on this Chrome and Apple Silicon combination the probe passed and execution trapped. The vendored worker was patched so the probe always reports relaxed SIMD as unavailable, falling back to the mature SIMD core. Separately, `chrome://extensions/shortcuts` showed the Mac shortcut (`Cmd+Shift+F`) as “Not set”; that was only explained in Phase 16.

Phase 16 — Release hardening, verified in real Chrome.

A review before publishing to the Chrome Web Store added, for the first time, an automated end-to-end test that loads the *packaged* extension into a throwaway Chrome profile and drives it through the DevTools protocol [15]. It found that the core feature had never worked: Chrome silently refuses to navigate a website tab to an extension’s `blob:` URL, so the tab never switched to the searchable copy. Copies are now served from the extension’s own origin by the service worker (decision 4). The same review and tests found and fixed:

- OCR collapsing into noise on sparse pages: the contrast stretch’s dark point fell inside the background, amplifying JPEG noise up to 127×; the stretch gain is now capped (Section 7).
- A release zip that omitted the `offscreen/` folder, because the packaging script still listed the deleted viewer.
- The Mac shortcut: Chrome leaves the suggested `Cmd+Shift+F` unassigned (`chrome.commands.getAll()` reports no shortcut), so the default is now `Control+Shift+F`, making the shortcut `Ctrl+Shift+F` on every platform.
- PDFs whose URL lacks “.pdf” were ignored; the extension now accepts any web or file URL and checks the file’s %PDF- signature.
- Danish letters were stripped from word edges (“på” was stored as “p”), because word trimming only recognized `a-z`; it now uses Unicode letter classes.
- OCR results keyed by URL could be applied to a changed file; the cache now keys by a SHA-256 of the content and prunes itself (decision 6).
- Silent failures: encrypted PDFs, non-PDF pages, and disabled file access now show a red badge with an explanation; the success badge is set only after the copy loads, because Chrome clears a tab’s badge when it navigates.
- Unbounded resources: pdf.js documents are destroyed after each job and idle OCR workers are released (decision 7); unused Tesseract builds were removed, shrinking the package from 22.5 MB to 8.5 MB.
- Store requirements: a manifest description within 132 characters, a privacy policy, and listing material.

Thought: unit-level checks had proved the pipeline; only driving the real browser proved the product.

6.3 Plan versus Reality

The original instinct that the *overlay* would be the hardest part was correct—coordinate alignment, the native-versus-image de-duplication, and the measure-and-scale fitting absorbed the most iteration. The estimate was wrong in its *shape*, though: “OCR” looked like one tidy phase but became the project’s centre of gravity, because the headline promise (find *any* text in *any* image) is exactly the kind of goal that surfaces a long tail of real-world failure modes—rotation, tables, multiple languages, blur, low contrast, small type. Each was its own phase, and each was reached by shipping the previous one and watching where it still fell short.

The plan was also wrong about where the project would end. It assumed a custom viewer throughout; in the end the custom viewer, its overlay, and its find bar were all deleted, and the coordinate work moved from the DOM into the PDF itself. The hardest late problems were

not algorithms but *platform behaviour*: what Chrome lets an extension do with its sandboxed viewer, with blob URLs, with keyboard shortcuts, and with toolbar badges. None of these showed up in tests of the pipeline alone; each surfaced only when the real browser was in the loop.

7 Implementation

This section walks through the pipeline stage by stage. Code excerpts are drawn from the extension’s source and lightly trimmed for space.

7.1 Extension Structure and Activation

The extension is a Manifest V3 package with no content scripts. Its content security policy permits `wasm-unsafe-eval` so the bundled Tesseract WebAssembly core can run, and the OCR engine, language data, PDF.js and pdf-lib are all bundled—no CDN fetch occurs at runtime.

Activation is on demand (decision 3). Two entry points—the toolbar icon (`action.onClicked`) and a keyboard shortcut (`commands.onCommand`, `Ctrl+Shift+F` on every platform)—run one handler (Listing 1). On a searchable copy it restores the original, whose URL is carried in the copy’s `src` parameter. Otherwise it starts the offscreen document and sends it the job, including the copy URL and cache key the service worker has chosen, so that the naming of copies lives in one place.

```

1  async function toggle(tab) {
2    // Showing a searchable copy? Restore the original PDF.
3    if (isCopyUrl(tab.url)) {
4      const original = originalOf(tab.url);
5      await dropSwap(tab.id);
6      await deleteCopy(tab.url);
7      if (original) chrome.tabs.update(tab.id, { url: original });
8      return;
9    }
10   if (!isFetchableUrl(tab.url)) return badgeError(tab.id, "open a PDF in
      this tab first");
11
12   badge(tab.id, "...", "Making this PDF searchable...");
13   await ensureOffscreen();
14   const url = tab.url.split("#")[0];
15   const copyUrl = newCopyUrl(url);    //
      chrome-extension://<id>/searchable/<uuid>/<name>?src=<url>
16   chrome.runtime.sendMessage({ type: "fii-process", tabId: tab.id, url,
17     lang: await getLang(), copyUrl, cache: COPY_CACHE, cacheKey:
      copyKey(copyUrl) });
18 }
19 chrome.action.onClicked.addListener((tab) => toggle(tab));
20 chrome.commands.onCommand.addListener((cmd, tab) => { if (cmd ===
      "toggle-search") toggle(tab); });

```

Listing 1. On-demand activation: one handler for the toolbar icon and the shortcut. It restores the original when the tab shows a copy, and otherwise starts a job (`service-worker.js`).

Local file:// PDFs additionally require the per-extension “Allow access to file URLs” setting; if it is off, the handler shows a red badge explaining this and opens the extension’s details page, where the toggle lives.

7.2 Download, Validation, and Identity

The offscreen document downloads the PDF again from the tab’s URL with `credentials: "include"`, so that PDFs on sites the user is signed in to return the same file the tab shows. The trigger accepts *any* web or file URL, because many PDFs are served from addresses without “.pdf” (for example `arxiv.org/pdf/2111.03017` or `download.php?id=7`). Validation therefore happens on the bytes: a PDF must contain the signature `%PDF-` within its first kilobyte. A page that returns HTML is reported as “not a PDF (or a sign-in page)”.

The bytes are then loaded twice, by PDF.js (for analysis and rendering) and by pdf-lib (for writing), while `crypto.subtle.digest` computes their SHA-256 in parallel. If either library reports encryption, the job ends with a clear message: pdf-lib cannot write into encrypted files. pdf-lib’s `EncryptedPDFError` is transpiled ES5, so `instanceof` does not recognize it and the check matches its message instead. The PDF.js loading task is destroyed in a `finally` block, because the offscreen document lives all session and would otherwise keep every processed document in memory.

7.3 Text Detection

For each page the processor first calls `getTextContent()`. OCR runs when the page paints raster images *or* when no text is extractable at all (a page may be a scan, or have its text converted to vector outlines—both invisible to `getTextContent()`). Image detection inspects the operator list:

```

1  const IMAGE_OPS = new Set([
2    pdfjsLib.OPS.paintImageXObject,
3    pdfjsLib.OPS.paintImageMaskXObject,
4  ]);
5  async function pageHasImages(page) {
6    const opList = await page.getOperatorList();
7    return opList.fnArray.some((fn) => IMAGE_OPS.has(fn));
8  }

```

Listing 2. Image detection: scans the PDF.js operator list for raster paint opcodes to decide whether a page needs OCR (`processor.js`).

If image detection fails it *fails open*—assuming images are present—on the principle that an unnecessary OCR pass is cheaper than silently missing text.

7.4 OCR Rendering and Preprocessing

Pages that need OCR are rendered to an offscreen canvas about 3000px wide (≈ 360 DPI for a Letter page), with PDF.js’s `print` intent so that rendering is not paced by `requestAnimationFrame`, which never fires in a hidden document. The canvas is released immediately after recognition.

Before recognition, the image is converted to grayscale and preprocessed to attack the two hardest cases, low contrast and blur:

1. **Gain-capped contrast stretch.** With P_2 and P_{98} the 2nd and 98th percentiles of the luminance histogram, every gray level v is remapped as

$$v' = 255 - 255 \frac{P_{98} - v}{R}, \quad R = \max(R_{\min}, P_{98} - P_2), \quad R_{\min} = 96. \quad (5)$$

When the percentiles are far apart this is the classic stretch ($P_2 \mapsto 0$, $P_{98} \mapsto 255$). The floor $R_{\min}$ matters on mostly blank pages: when text covers less than 2% of the pixels, P_2 is itself background. On the first test pages P_2 was 233–238 against $P_{98} = 240$, so the uncapped stretch multiplied contrast by up to $127\times$, turned faint JPEG noise into black speckle, and made one page unreadable. Anchoring at P_{98} keeps the background white, and the floor limits the gain to $255/96 \approx 2.7$.

2. **Unsharp mask.** Edges are re-sharpened as $\text{sharp} = g + \alpha (g - g_{\text{blur}})$ with $\alpha = 1$, where g_{blur} comes from a separable box blur implemented with a sliding-window sum, so its cost is $O(n)$ independent of radius.

Tesseract performs its own binarization afterward, so this stage only needs to hand it a clean, high-contrast grayscale image.

7.5 OCR Engine Orchestration

Recognition runs through a Tesseract.js *scheduler* that distributes pages across a pool of Web Workers. The worker count scales with CPU cores but is capped, because each worker loads its own copy of the language model:

$$N_{\text{workers}} = \max(1, \min(4, \text{hardwareConcurrency} - 1)). \quad (6)$$

The processor keeps N_{workers} pages in flight at once. JavaScript is single-threaded, so pdf-lib edits to different pages interleave safely; only the rendering and recognition awaits overlap. Workers use *sparse text* page segmentation, which finds table cells, captions, and scattered text far better than the default layout analysis. Three language modes are supported—**eng**, **dan**, and **eng+dan**—with one worker pool per language. Sixty seconds after the job queue empties, all pools are terminated to free their memory; they are recreated on the next job.

7.6 Post-Processing for Findability

Because native find performs exact substring matching, fuzzy matching is impossible at search time; the post-processing stage improves the *stored* text instead. It drops near-zero-confidence noise, trims non-letter junk from word edges (with Unicode letter classes, so Danish words keep their æ , ø and å), and emits a *correction variant* for words that look like digit/letter confusions (**he11o** $\rightarrow$ **hello**, **ca5h** $\rightarrow$ **cash**). Tokens that are more than half digits are left alone as genuine numbers or codes:

```

1  const DIGIT_TO_LETTER = {
    "0": "o", "1": "l", "5": "s", "6": "b", "8": "b", "9": "g", "2": "z" };
2  out.push({ text, bbox: w.bbox }); // the word as
    recognized
3  const variant = correctionVariant(text); // null unless mostly
    letters + some digits
4  if (variant && variant !== text)
5    out.push({ text: variant, bbox: w.bbox, variant: true });
```

Listing 3. Emitting a correction variant for OCR tokens with likely digit/letter confusions (ocr-postprocess.js).

The earlier DOM viewer injected both forms and hid the variant from selection. Inside a real PDF there is no such distinction: two words at one spot would double Chrome’s match count and duplicate copied text. The processor therefore embeds a single form, preferring the corrected variant, since it is what the image most likely says.

7.7 Writing the Invisible Layer

Each recognized word is mapped into PDF user space as described in Section 5.3 and de-duplicated against the page's existing text. The survivors are appended to the page's content stream as one invisible text object each (Listing 4). The whole block is wrapped in `q/Q` so its text state cannot leak into anything drawn later. Helvetica's WinAnsi encoding covers English and Danish; a word containing a character it cannot encode is skipped rather than failing the page.

```

1  for (const wd of words) {
2    const encoded = font.encodeText(wd.text);
3    const natural = font.widthOfTextAtSize(wd.text, wd.h);
4    const squeeze = Math.max(1, Math.min(1000, (wd.w / natural) * 100));
      // Tz, percent
5    const cos = Math.cos(wd.angle), sin = Math.sin(wd.angle);
6    ops.push(
7      PDFLib.beginText(),
8      PDFLib.setTextRenderingMode(PDFLib.TextRenderingMode.Invisible),
      // 3 Tr
9      PDFLib.setFontAndSize(fontKey, wd.h),
10     PDFLib.setCharacterSqueeze(squeeze),
11     PDFLib.setTextMatrix(cos, sin, -sin, cos, wd.x, wd.y),
12     PDFLib.showText(encoded),
13     PDFLib.endText(),
14   );
15 }
16 page.pushOperators(PDFLib.pushGraphicsState(), ...ops,
    PDFLib.popGraphicsState());

```

Listing 4. Writing one invisible, width-matched text object per word with pdf-lib (embed.js).

The document is then saved with its metadata untouched, and the bytes are stored in Cache Storage as an `application/pdf` response under the key the service worker supplied.

7.8 Caching

Recognized words are normalized to the unit square (Eq. (1)) and stored in IndexedDB under `<sha256>::<lang>::<page>`. Because the key is the file's content hash, the same PDF reached through two different URLs is recognized once, and a different file at a familiar URL is never matched to stale words. Empty results are cached too, so text-only pages are not re-examined. Each record carries a *format version*, bumped whenever the OCR pipeline changes, and a last-used timestamp. A cache hit refreshes the timestamp, and after every job the oldest records are deleted until at most 2,000 pages remain.

7.9 Serving and Cleaning Up the Copy

When the job is done, the service worker checks that the tab still shows the same PDF (the user may have moved on), records which copy the tab shows, and navigates it to the copy URL. Chrome requests that URL from the extension, and the service worker's `fetch` handler answers it (Listing 5). Chrome sees an `application/pdf` response and opens its own viewer; because the path ends in the original file name, the tab title is unchanged. The success badge is set only after the tab finishes loading, because Chrome clears a tab's badge whenever it navigates.

```

1  self.addEventListener("fetch", (event) => {
2    const url = new URL(event.request.url);
3    if (url.origin !== self.location.origin ||
        !url.pathname.startsWith("/searchable/")) return;

```

```

4   event.respondWith(serveCopy(url));
5 });
6 async function serveCopy(url) {
7   const hit = await (await
8     caches.open(COPY_CACHE)).match(COPY_KEY_ORIGIN + url.pathname);
9   if (hit) return hit;
10  // The copy is gone (e.g. the browser restarted): fall back to the
11  // original PDF.
12  const src = url.searchParams.get("src");
13  if (src && isFetchableUrl(src)) return Response.redirect(src, 302);
14  return new Response("This searchable copy is no longer available.", {
15    status: 404 });
16 }

```

Listing 5. Serving searchable copies at the extension’s own URLs. Cache Storage only accepts http(s) keys, so copies are stored under a placeholder origin with the same path (`service-worker.js`).

Copies are deleted when the user restores the original, when their tab navigates elsewhere or closes, and when the browser starts. A tab that returns to a deleted copy—through session restore or the browser’s history—simply redirects to the original PDF.

8 Results and Evaluation

The system is evaluated by an automated end-to-end test of the *packaged* extension in real Chrome, complemented by measurements of OCR accuracy, timing, and visual fidelity on a small set of test documents. The corpus is small and synthetic, so the accuracy figures characterize behaviour rather than serve as a benchmark (Section 9).

8.1 Method: End-to-End Testing in Chrome

The test (`npm run test:e2e`) builds the release zip, unpacks it, and launches Google Chrome with a throwaway profile, connected through the DevTools protocol over a pipe [15]. It loads the extension with `Extensions.loadUnpacked`, attaches to the extension’s service worker, and invokes the same handler the toolbar icon calls. A local HTTP server plays the web: it serves image-only PDFs, a PDF at a URL without “.pdf”, a PDF behind a sign-in cookie, a URL whose file changes between requests, a non-PDF page, and two encrypted PDFs. The fixtures are US Letter pages rendered at 300 DPI and saved as image-only PDFs, like a scanner’s output: a dense page with a paragraph, small print and a bordered table; a sparse receipt-like page; and a delivery note with a table. Each check observes what Chrome actually did—the tab’s URL and title, the toolbar badge—and reads the copy back out of Cache Storage to extract its text with an independent `PDF.js` instance.

8.2 Functional Outcomes

All 24 checks pass on the final build. Table 5 groups them by scenario.

The suite’s most important result came before it passed. Its first run showed that the shipped design never completed the tab switch—OCR finished and the copy existed, but Chrome would not navigate the tab to a `blob:` URL. No earlier test could have noticed, because none involved Chrome’s own tab navigation. The redesign of decision 4 followed directly.

8.3 OCR Accuracy

On the dense page, the copy’s text layer contains all 74 words, including the small print and every table cell; one character was misread (“ZX-4471” as “7X-4471”). The sparse page and

Table 5. End-to-end scenarios exercised against the packaged extension in Chrome. All checks pass on the final build.

Scenario	Verified behaviour
Shortcut	Chrome actually assigns the shortcut (Ctrl+Shift+F ; Control+Shift+F on Mac)
Scanned PDF	Tab switches to the copy in Chrome’s viewer; the copy’s text layer contains the page’s words; page count and tab title unchanged; success badge shown
Reload, restore	Reloading the copy shows the copy; invoking again restores the original URL and deletes the copy
URL without “.pdf”	Processed; tab title (“Quarterly Report”) unchanged; reuses the OCR cache from the same file at another URL
Expired copy	With the copy deleted (as after a restart), reloading lands on the original PDF
Signed-in PDF	A cookie-protected PDF is downloaded and processed with the signed-in document’s words; closing the tab deletes its copy
Changed file, same URL	The new file is processed and contains only its own words
Encrypted PDFs	Both an open-password and a permissions-only PDF report a clear error
Non-PDF page	Reports “not a PDF” instead of failing silently
Local file	Processed and restored to its file:// URL when file access is allowed
Idle worker release	After 65s idle, the next document is still recognized correctly

the delivery note were recognized word for word, including their tables. The gain cap of Eq. (5) was decisive on sparse pages. On the first, oversized test pages, the uncapped stretch turned one page into unrelated fragments and broke words in another’s headline (“Quarterly Summary” for “Quarterly Summary”); with the cap, both read correctly.

Two observations temper these numbers. The fixtures are clean renders, kinder than real scans with skew, noise and uneven lighting. And the 3000px OCR resolution is tuned for ordinary type sizes: on test pages with unusually large text, Tesseract skipped bordered table cells that it read at half the resolution.

8.4 Speed and Size

Measured by the test harness on the development Mac (at 0.5s polling granularity), a one-page scan goes from the user’s gesture to the searchable copy on screen in about 2–2.5s, including starting the OCR workers. The same file again—even from another URL—takes about 0.5s, as OCR is skipped, and a new document after the workers have been released takes about 1s. Multi-page documents scale with the page count divided by the number of workers.

The release package is 8.5MB. Shipping only the two Tesseract builds that can actually load (of the six previously bundled, each in three file formats) cut it from 22.5MB.

8.5 Visual Fidelity

Invisible text paints nothing, so the copy should look exactly like the original. When the sandwich design was introduced, the original and the copy were rendered and compared pixel by pixel: 0 of 2,298,825 pixels differed. The end-to-end suite additionally checks that the page count and the tab title are unchanged. Everything the user sees—toolbar, find bar, highlights, context menu—is Chrome’s own.

8.6 Component Complexity

Table 6 summarizes the relative implementation complexity of each component after the redesign. Complexity moved from the DOM overlay, which no longer exists, to the integration with Chrome.

Table 6. Relative complexity by component.

Component	Complexity
PDF analysis and rendering (PDF.js)	Medium
OCR pipeline and preprocessing (Tesseract.js)	Medium
Coordinate mapping into PDF space	High
Invisible-text embedding (pdf-lib)	Medium
Chrome integration (offscreen, tab switch, serving copies)	High
Caching and resource lifecycle	Medium

8.7 Discussion

The results support the project’s central thesis in its strongest form: by turning the document into real text, the system inherits not only the browser’s search but its entire PDF experience, and adds no interface of its own. The engineering effort concentrated where the real difficulty lay—placing words precisely in PDF space, keeping a CPU-heavy workload in the background, and fitting into what the browser allows an extension to do. The last of these is also the lesson of the evaluation: only testing the packaged extension in the real browser revealed whether the product worked at all.

9 Limitations and Future Work

9.1 Limitations

OCR is not perfect.

Recognition errors propagate into search. Because native find is exact, a misread character can make a word unfindable by its true spelling. Correction variants fix the most common digit/letter confusions but not the general case.

The whole document is processed before the switch.

The copy appears only when every page is done, so large scans take correspondingly longer (the badge shows progress meanwhile). Caching makes later visits fast.

Encrypted PDFs cannot be processed.

pdf-lib cannot write into encrypted files, so password-protected and permission-restricted PDFs are reported and left as they are.

Latin character set.

The invisible text uses the standard Helvetica font with WinAnsi encoding, which covers English and Danish but not, for example, Greek, Cyrillic, or CJK scripts.

The PDF must be downloadable again.

The extension re-fetches the tab’s URL. PDFs produced by a form submission or served from one-time links cannot be fetched a second time, and PDFs shown inside another site’s viewer (rather than in Chrome’s) are not PDF tabs at all.

The address bar changes.

While the copy is shown, the address bar reads `chrome-extension://.../searchable/...`. The tab title, the content, and every control are unchanged, but this is the one visible sign of the switch.

Broad host access.

To download a PDF from whatever site it is on, the extension requests access to all sites. It uses that access only when the user invokes it, but the install prompt and the store review treat it as broad.

Device-dependent performance.

Throughput depends on the number of CPU cores, since the worker pool scales with available concurrency.

Discoverability.

Because the extension deliberately leaves Chrome's PDF viewer alone and activates on demand (decision 3), it has no surface on which to advertise itself. A first-time user must learn the shortcut or the toolbar icon from the store listing or the icon's tooltip.

Limited evaluation.

The accuracy results come from a few clean, synthetic scans, and the end-to-end tests run on one machine with one Chrome version.

9.2 Future Work

- **Faster time to searchable.** Process pages in reading order and offer a partial copy early for long documents.
- **Wider scripts and languages.** Embed a subsetted Unicode font for the invisible text, bundle more language models, and detect the script automatically.
- **Higher OCR accuracy.** Dictionary-based correction, confidence-weighted variants, automatic skew, and OCR at more than one scale so that unusually large type is recognized as reliably as body text.
- **Narrower permissions.** Request access to a site only at the moment the user invokes the extension there, once it is verified that this also covers downloading the PDF.
- **Quantitative evaluation.** A benchmark of real scanned documents measuring character and word error rates and per-page latency, to turn the observations of Section 8 into reproducible numbers.
- **Images on ordinary web pages.** Extending the approach to text inside `<img>` elements on regular pages, which would need a different delivery mechanism than a PDF copy.

10 Conclusion

FIND IN IMAGES demonstrates that the most effective way to make a scanned document searchable in the browser is not to build a search engine, but to *make the document into real text* and let the browser do what it already does well. In its final form, the extension adds no interface at all. On demand, it recognizes the text in a PDF's images on the user's own machine, writes it into a copy of the file as an invisible layer, and lets Chrome's own viewer and Chrome's own find bar do the rest. The original file is never modified, and the copy looks exactly like it.

Reaching that form took two reversals. The project began with a custom viewer that overlaid invisible DOM text on PDF.js renderings, which required a careful coordinate pipeline, measure-and-scale fitting, and a custom find bar. The first reversal made the viewer opt-in, because replacing Chrome's PDF experience for every document cost more than it gave. The second removed the viewer entirely, because any imitation of Chrome's interface was still not Chrome's interface. The geometric work did not disappear; it moved into the PDF, where each word is placed in user space at the right size, angle and width for Chrome's own highlight to cover it.

The last phase supplied the project's most practical lesson. Every component had been tested, yet the product did not work: Chrome would not open the copy from where it was stored. The bug became visible only when the packaged extension was driven through the real browser. That test now protects the release, and its fixes—serving copies from the extension's origin, content-addressed caching, a contrast stretch that cannot amplify noise, and bounded resources—are what make the extension ready for others to use.

More broadly, the project illustrates a reusable principle for client-side engineering: bridge a new capability (OCR) into an existing, battle-tested substrate (the PDF format and the browser's viewer) rather than re-implementing that substrate—and verify the result where users will actually meet it.

References

- [1] Adobe Systems, “Document management — portable document format — part 1: PDF 1.7 (ISO 32000-1).” ISO standard, 2008.
- [2] J. R. Barlow and OCRmyPDF Contributors, “OCRmyPDF: Adds an OCR text layer to scanned PDF files.” <https://github.com/ocrmypdf/OCRmyPDF>, 2024. Accessed 2026.
- [3] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. Ninth International Conference on Document Analysis and Recognition (ICDAR)*, pp. 629–633, IEEE, 2007.
- [4] Tesseract.js Contributors, “Tesseract.js: Pure JavaScript OCR for more than 100 languages.” <https://tesseract.js.org/>, 2024. Accessed 2026.
- [5] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [6] Mozilla, “PDF.js: A general-purpose, web standards-based platform for parsing and rendering PDFs.” <https://mozilla.github.io/pdf.js/>, 2024. Accessed 2026.
- [7] A. Dillon, “pdf-lib: Create and modify PDF documents in any JavaScript environment.” <https://pdf-lib.js.org/>, 2021. Accessed 2026.
- [8] Google, “Chrome extensions: Manifest V3 overview.” <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>, 2024. Accessed 2026.
- [9] MDN Web Docs, “FetchEvent: Service worker API.” <https://developer.mozilla.org/en-US/docs/Web/API/FetchEvent>, 2024. Accessed 2026.
- [10] Google, “chrome.offscreen api reference.” <https://developer.chrome.com/docs/extensions/reference/api/offscreen>, 2024. Accessed 2026.
- [11] MDN Web Docs, “IndexedDB API.” https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API, 2024. Accessed 2026.
- [12] MDN Web Docs, “Cache API.” <https://developer.mozilla.org/en-US/docs/Web/API/Cache>, 2024. Accessed 2026.
- [13] MDN Web Docs, “SubtleCrypto.digest(): Web crypto API.” <https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/digest>, 2024. Accessed 2026.
- [14] Chromium blink-dev, “Implement and ship: Blob URL partitioning: Fetching/navigation.” <https://groups.google.com/a/chromium.org/g/blink-dev/c/erVBugcYwRc>, 2025. Accessed 2026.
- [15] Google, “Chrome DevTools protocol.” <https://chromedevtools.github.io/devtools-protocol/>, 2024. Accessed 2026.